

Document number: P2601R1
Date: 2022-07-11
Reply-to: Justin Cooke <jgc@cems.de>
Target audience: EWG, CWG

Make redundant empty angle brackets optional

Proposal: To make redundant empty angle brackets optional for class and variable template argument lists.

Revision history:

R0: 2022-06-11

R1: (this revision):

- (i) description amended to clarify that it refers both to class and variable templates;
- (ii) description and wording amended to clarify the distinction between defaulted and deduced template arguments;
- (iii) additional examples included of where empty angle brackets can be omitted without effect, and a counter-example where their presence/absence is significant

Motivation: Additional template parameters with default arguments can be added to a class template declaration in a header file without breaking code that uses the template. For example, early versions of `std::vector` took just one parameter, the value type; the allocator parameter was added later, with a default argument. However, that upgrade pathway is currently only open where there is already at least one template parameter. It is not currently possible to replace a non-template class declaration by a class template, even with default arguments for all parameters, without breaking most uses of the class, because the syntax for using class templates requires empty angle brackets in most circumstances, even where their omission would not result in ambiguity. Since the introduction (in C++17) of CTAD, angle brackets can be omitted when defining objects, but they are still required when declaring pointers, references, arrays, external objects, function parameters, class members, base classes, or template type parameters. The following examples illustrate the current situation:

```
template <typename T = int> class C {};  
C c;                                //OK (#1)  
extern C x;                          //error: need C<>  
C *p;                               //error: need C<>  
C a[2];                             //error: need C<>  
C &r = c;                           //error: need C<>  
struct S { C c; };                  //error: need C<>  
struct D : C {};                    // error: need C<>  
std::vector<C> v;                   // error: need C<>  
void foo(C c);                      // error: need C<>
```

The empty angle brackets are currently optional only in case (#1) (CTAD). No ambiguity would arise from making them optional in all the above cases, because names of class or variable templates must be unique within a scope (13.1 Note 5). This change will allow class library maintainers to upgrade existing class definitions to templates without breaking user code.

Description: The argument list for a class or variable template can be empty when all arguments are defaulted or all arguments are defaulted except for an empty argument for a

trailing parameter pack. This proposal makes optional the angle brackets enclosing an empty class or variable template argument list, except where the brackets are needed for disambiguation. This is the case for in initializing declarations, where use of a template name without an argument list activates class template argument deduction (CTAD, 12.2.2.9). This proposal does not affect CTAD, but an example is included to illustrate how the presence of angle brackets suppresses CTAD.

The proposal does not affect function templates. However, in declaring angle brackets to be optional for empty template argument lists in general, the wording needs to mention that the brackets are still required to disambiguate a function template from a non-template function with the same name and scope.

The proposal does not affect template *parameter* lists, where empty angle brackets disambiguate explicit full specialization declarations (13.9.4) from explicit instantiation declarations (13.9.3).

Effect on existing code: None. Code using the proposed feature is ill-formed under C++20 and earlier standards.

Wording: (edits to N4910)

13.4.1 [temp.arg.general]

...

5 When template argument packs or default *template-arguments* are used, a *template-argument* list can be empty. In that case the empty `<>` brackets **may be omitted** ~~shall still be used as the template argument list~~ **except where needed to disambiguate a class template specialization with an empty argument list from a template name used as a placeholder for a deduced class type (9.2.9.7) or to disambiguate an invocation of a function template specialization from an invocation of a non-template function with the same name (13.10.2).**

[Example 5 :

```
template<class T = char> class String;
String<>* p; // OK, String<char>
String* q; // syntax error OK, empty brackets <> omitted

template<class ... Elements> class Tuple;
Tuple<>* t; // OK, Elements is empty
Tuple* u; // syntax error OK, empty brackets <> omitted

template <class T = int> struct C { C(T); };
std::vector<C> v; //C<int>
struct S { C c; }; //C<int>
struct D : C {}; // C<int>
void foo(C c); // C<int>
C x = 1.0; //deduced to C<double> (see 12.2.2.9)
C<> y = 1.0; //C<int>, initializer converted to int
```

—end example]